EDA ASSIGNMENT

Extracted Python code from the uploaded Jupyter Notebook

Notebook cells: 21 | Code cells: 11

Code Cell 2

```
# ========================================
# COLLEGE PLACEMENT &amp;amp; SKILL ANALYSIS
# ========================================

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from google.colab import files
from IPython.display import display

sns.set_theme(style="whitegrid")

print("College Placement and Skill Analysis")
print("Libraries imported successfully!")
```

Code Cell 4

```
# Upload your student placement CSV file
uploaded = files.upload()

if uploaded:
    filename = next(iter(uploaded))
    df = pd.read_csv(filename)
    print("Dataset loaded:", filename)

else:
    print("No file uploaded.")

# Required columns for the project:
# Student_ID, CGPA, Technical_Skills, Communication_Skills,
# Aptitude_Score, Internship, Projects, Placement_Status

if 'df' not in globals():
    df = pd.DataFrame({
        'Student_ID': range(1, 201),
        'CGPA': np.round(np.random.uniform(5.0, 10.0, 200), 2),
        'Technical_Skills': np.random.randint(30, 101, 200),
        'Communication_Skills': np.random.randint(30, 101, 200),
        'Aptitude_Score': np.random.randint(30, 101, 200),
        'Internship': np.random.choice(['Yes', 'No'], 200),
        'Projects': np.random.randint(0, 7, 200)
    })

    # Demonstration-only placement labels
    score = (
        df['CGPA'] * 7
        + df['Technical_Skills'] * 0.25
        + df['Communication_Skills'] * 0.15
        + df['Aptitude_Score'] * 0.20
        + (df['Internship'] == 'Yes') * 8
        + df['Projects'] * 1.5
    )

    df['Placement_Status'] = np.where(
        score &gt;= 65, 'Placed', 'Not Placed'
    )

print("\nDataset Preview:")
display(df.head(10))

print("\nDataset Shape:", df.shape)
print("\nColumn Names:")
print(df.columns.tolist())
```

Code Cell 6

```
# ========================================
# DATA CLEANING
```

```
# ========================================

print("Missing Values Before Cleaning:")
display(df.isnull().sum())

print("Duplicate Rows:", df.duplicated().sum())

df = df.drop_duplicates().copy()

# Convert numerical columns to numeric format
numeric_cols = [
    'CGPA',
    'Technical_Skills',
    'Communication_Skills',
    'Aptitude_Score',
    'Projects'
]

for col in numeric_cols:
    if col in df.columns:
        df[col] = pd.to_numeric(df[col], errors='coerce')

# Fill missing numeric values with the median
for col in numeric_cols:
    if col in df.columns:
        df[col] = df[col].fillna(df[col].median())

# Fill missing categorical values
categorical_cols = ['Internship', 'Placement_Status']

for col in categorical_cols:
    if col in df.columns:
        df[col] = df[col].fillna('Unknown')

print("\nMissing Values After Cleaning:")
display(df.isnull().sum())

print("\nStatistical Summary:")
display(df.describe(include='all'))
```

Code Cell 8

```
# ========================================
# PLACEMENT STATISTICS
# ========================================

if 'Placement_Status' not in df.columns:
    raise ValueError("Dataset must contain Placement_Status.")

placement_counts = df['Placement_Status'].value_counts()

total_students = len(df)

placed_students = (
    df['Placement_Status'].astype(str).str.strip().str.lower()
    == 'placed'
).sum()

not_placed_students = total_students - placed_students

placement_percentage = (
    placed_students / total_students * 100
    if total_students &gt; 0 else 0
)

print("Total Students:", total_students)
print("Placed Students:", placed_students)
print("Other Placement Statuses:", not_placed_students)
print(f"Placement Percentage: {placement_percentage:.2f}%")

plt.figure(figsize=(7, 5))
sns.countplot(
    data=df,
    x='Placement_Status',
    hue='Placement_Status',
    legend=False
)

plt.title("College Placement Status Analysis")
plt.xlabel("Placement Status")
plt.ylabel("Number of Students")
plt.tight_layout()
```

```
plt.show()
```

Code Cell 10

```
# ========================================
# SKILL ANALYSIS
# ========================================

skill_columns = [
    'Technical_Skills',
    'Communication_Skills',
    'Aptitude_Score'
]

available_skills = [
    col for col in skill_columns if col in df.columns
]

if available_skills:
    skill_averages = df[available_skills].mean()

    print("Average Student Skill Scores:")
    display(skill_averages.to_frame("Average Score"))

    plt.figure(figsize=(9, 5))

    sns.barplot(
        x=skill_averages.index,
        y=skill_averages.values
    )

    plt.title("Average Student Skills Analysis")
    plt.xlabel("Skill Category")
    plt.ylabel("Average Score")
    plt.xticks(rotation=15)
    plt.tight_layout()
    plt.show()
```

Code Cell 12

```
# ========================================
# PLACEMENT VS SKILLS
# ========================================

if available_skills and 'Placement_Status' in df.columns:

    comparison = df.groupby(
        'Placement_Status'
    )[available_skills].mean()

    print("Average Skills by Placement Status:")
    display(comparison.round(2))

    comparison.plot(
        kind='bar',
        figsize=(10, 6)
    )

    plt.title("Skills Comparison by Placement Status")
    plt.xlabel("Placement Status")
    plt.ylabel("Average Skill Score")
    plt.xticks(rotation=0)
    plt.legend(title="Skills")
    plt.tight_layout()
    plt.show()
```

Code Cell 14

```
# ========================================
# CGPA ANALYSIS
# ========================================

if 'CGPA' in df.columns:

    plt.figure(figsize=(9, 5))

    sns.boxplot(
        data=df,
        x='Placement_Status',
        y='CGPA',
        hue='Placement_Status',
```

```
        legend=False
    )

    plt.title("CGPA Distribution by Placement Status")
    plt.xlabel("Placement Status")
    plt.ylabel("CGPA")
    plt.tight_layout()
    plt.show()

    cgpa_analysis = df.groupby(
        'Placement_Status'
    )['CGPA'].agg(['mean', 'median', 'min', 'max'])

    display(cgpa_analysis.round(2))
```

Code Cell 16

```
# ========================================
# CORRELATION ANALYSIS
# ========================================

correlation_columns = [
    col for col in [
        'CGPA',
        'Technical_Skills',
        'Communication_Skills',
        'Aptitude_Score',
        'Projects'
    ]
    if col in df.columns
]

if len(correlation_columns) &gt;= 2:

    correlation_matrix = df[correlation_columns].corr()

    plt.figure(figsize=(9, 6))

    sns.heatmap(
        correlation_matrix,
        annot=True,
        cmap='coolwarm',
        fmt='.2f',
        vmin=-1,
        vmax=1
    )

    plt.title("Student Skills and Academic Performance Correlation")
    plt.tight_layout()
    plt.show()
```

Code Cell 18

```
# ========================================
# INTERNSHIP ANALYSIS
# ========================================

if 'Internship' in df.columns:

    internship_counts = pd.crosstab(
        df['Internship'],
        df['Placement_Status']
    )

    print("Internship vs Placement:")
    display(internship_counts)

    internship_counts.plot(
        kind='bar',
        stacked=True,
        figsize=(8, 5)
    )

    plt.title("Internship Experience vs Placement Status")
    plt.xlabel("Internship Experience")
    plt.ylabel("Number of Students")
    plt.xticks(rotation=0)
    plt.tight_layout()
    plt.show()
```

Code Cell 20

```
# ========================================
# MACHINE LEARNING MODEL
# ========================================

from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix,
    ConfusionMatrixDisplay
)

target = 'Placement_Status'

feature_candidates = [
    'CGPA',
    'Technical_Skills',
    'Communication_Skills',
    'Aptitude_Score',
    'Internship',
    'Projects'
]

features = [c for c in feature_candidates if c in df.columns]

model_df = df[features + [target]].dropna().copy()

X = model_df[features]
y = model_df[target].astype(str)

if y.nunique() &lt; 2:
    print("The dataset needs at least two placement-status classes.")

elif y.value_counts().min() &lt; 2:
    print("Each class needs at least two records for a stratified split.")

else:
    numeric_features = X.select_dtypes(
        include=np.number
    ).columns.tolist()

    categorical_features = [
        c for c in X.columns if c not in numeric_features
    ]

    preprocessor = ColumnTransformer(
        transformers=[
            ('num', 'passthrough', numeric_features),
            ('cat', OneHotEncoder(handle_unknown='ignore'),
             categorical_features)
        ]
    )

    model = Pipeline([
        ('preprocessor', preprocessor),
        ('classifier', RandomForestClassifier(
            n_estimators=200,
            random_state=42,
            class_weight='balanced'
        ))
    ])

    X_train, X_test, y_train, y_test = train_test_split(
        X,
        y,
        test_size=0.2,
        random_state=42,
        stratify=y
    )

    model.fit(X_train, y_train)

    predictions = model.predict(X_test)

    print(
        f"Model Accuracy: "
        f"{accuracy_score(y_test, predictions) * 100:.2f}%"
    )
```

```
print("\nClassification Report:")
print(classification_report(y_test, predictions, zero_division=0))

ConfusionMatrixDisplay.from_predictions(
    y_test,
    predictions,
    xticks_rotation='vertical'
)

plt.title("Placement Prediction - Confusion Matrix")
plt.tight_layout()
plt.show()
```

Code Cell 21

Empty code cell